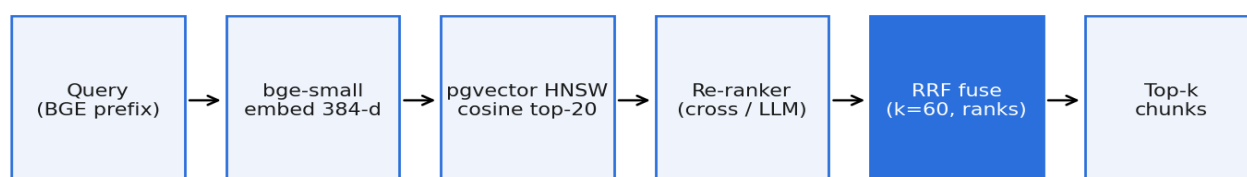


Search & Re-ranking: the full story

Architecture, evaluation method, every experiment (including the failures), results and lessons. Amaljith TA, 2026.

TL;DR. Plain vector search was already strong (MRR 0.876). A bigger cross-encoder used on its own made things worse (MRR 0.656). Fusing re-ranker ranks with vector ranks using Reciprocal Rank Fusion fixed that. The best setup, a small MiniLM cross-encoder with RRF, reached MRR 0.936 and lenient R@1 0.90 in about 1 s. A later chunk-boundary change moved MRR by about 0.05, the same size as the gains, which shows the 30-question eval is too small to separate close configurations. The next step is more questions plus bootstrap confidence intervals.

1. The pipeline



Documents are parsed (PDF → text, pages joined with blank lines), optionally cleaned of repeating footers, and split into 1,000-character chunks with 200-character overlap. Each chunk is embedded with **bge-small-en-v1.5** (384 dimensions) and stored in Postgres with **pgvector** behind an HNSW index. At query time the question gets the BGE query prefix, is embedded, and the nearest chunks are fetched by cosine distance (similarity = 1 – distance), filtered to documents with status 'ready'.

When re-ranking is on, retrieval fetches a larger candidate pool (20 for cross-encoders, 10 for LLM re-rankers), a re-ranker scores every (query, chunk) pair, and the result is either used alone or fused with the original vector order through RRF. Everything is config-driven (RERANK_ENABLED, RERANK_STRATEGY, RERANK_MODEL, candidate counts), so the eval can switch strategies from the command line.

Strategy	What it does	Cost
cross	Cross-encoder reads query+chunk together, outputs relevance	Local CPU, 0.5–6 s
llm_point	LLM scores each chunk 0–10 (pointwise)	1 LLM call per chunk
llm_list	LLM returns an ordering of all candidates (RankGPT-style), repaired for duplicates and missing query	1 LLM call per query
*_fused	Any of the above, then RRF with vector rank: score = $\sum 1/(60+\text{rank})$	Free on top

Why RRF uses ranks, not scores: cosine similarity lives around 0.6–0.8, cross-encoder logits are unbounded, LLM scores are 0–10. Adding them directly is meaningless; ranks are on a common scale. k=60 is the standard constant that stops the top-1 position from dominating.

2. How we measured it

Golden set: 37 hand-written questions on the University of London student handbook: 22 simple, 8 conditional, 7 unanswerable. Each answerable question has an evidence quote verified against the source. Ranking metrics use the 30 answerable ones.

Hit test (is_hit): a retrieved chunk counts as correct if it contains the normalised evidence exactly (*strict*) or covers ≥60% of its words (*lenient*, which tolerates evidence split across a chunk boundary).

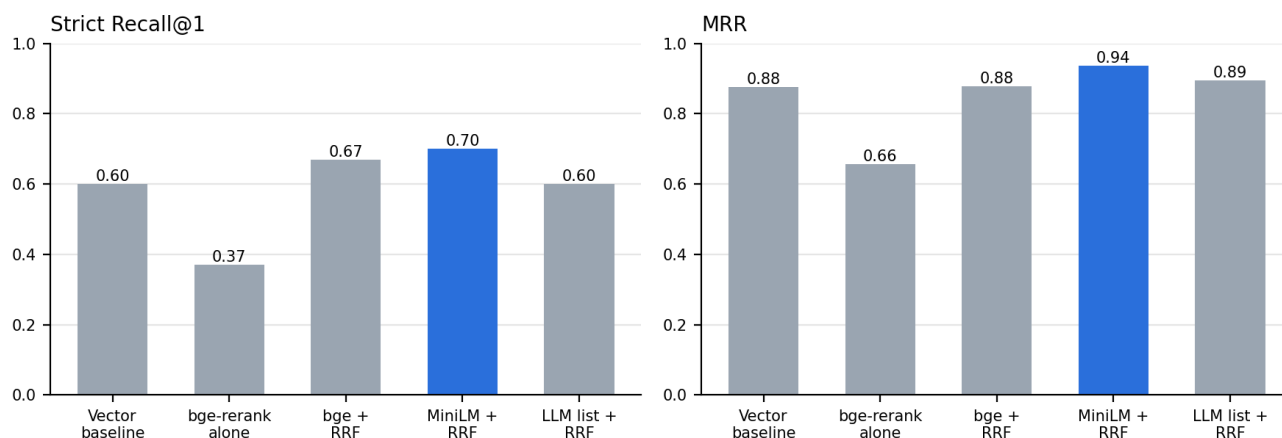
Metrics: Recall@1/5/10 (is the evidence in the top k?), MRR (1/rank of first hit, averaged), nDCG@10 (rewards hits near the top, binary relevance), per-slice reporting, latency per question, plus an LLM failure counter so silent API errors cannot hide inside a score.

Test isolation: 13 search tests plus 5 cleaning tests run inside rolled-back transactions with existing docs hidden, so real uploads never break the tests.

3. Results

Configuration	sR@1	R@1	sR@5	MRR	nDCG@10	Latency
Vector baseline	0.60	0.80	0.967	0.876	0.871	20 ms
bge-reranker-base alone (20 cand.)	0.37	0.47	0.93	0.656	0.729	5.8 s
bge-reranker-base + RRF	0.67	0.80	0.90	0.878	0.871	9.4 s
MiniLM-L-6 + RRF (best)	0.70	0.90	0.933	0.936	0.919	0.5–1.9 s
LLM listwise gpt-oss-120b + RRF	0.60	0.80	0.967	0.894	0.892	~20 s*
Clean chunks: vector baseline	0.567	0.767	0.967	0.861	0.854	20 ms
Clean chunks: MiniLM + RRF	0.60	0.80	0.967	0.886	0.892	~1 s

sR = strict recall, R = lenient recall. *LLM latency is mostly the free-tier throttle (20 s between calls on Groq), not model speed. 30 answerable questions, UoL handbook.



4. The story, experiment by experiment

4.1 Baseline: vector search is already good

Strict R@1 0.60 and MRR 0.876 at 20 ms. The right chunk was almost always in the top 5 (sR@5 0.967), so the headroom was in *ordering*, which is exactly what re-rankers claim to fix.

4.2 The bigger re-ranker made things worse

bge-reranker-base on its own dropped MRR from 0.876 to 0.656 and took 5.8 s per query. My first assumption was a bug. I ruled it out step by step: batched vs single-pair scores were identical (`check_batch.py`), and a sanity pair scored 0.997 vs 0.00004 for an irrelevant one, so the model was loaded and scoring correctly. Inspecting per-question rankings (`inspect_rerank.py`) showed the real cause: the model latched onto **lexical cues**. A chunk repeating phrases like 'first port of call' outranked the chunk that actually answered the question. The re-ranker was trained on web search data (MS MARCO-style), and handbook prose with repeated boilerplate fooled it.

Lesson: a model that wins on public benchmarks is not guaranteed to win on your documents. Measure before adopting.

4.3 Fusion rescues it: RRF

Rather than trusting the re-ranker alone, I fused its ranking with the vector ranking using RRF. A chunk must now look good to *both* systems to rise. bge + RRF recovered to MRR 0.878 with a better strict R@1 (0.67), but at 9.4 s it was too slow for the gain.

4.4 The small model wins: MiniLM + RRF

Swapping to the much smaller **ms-marco-MiniLM-L-6-v2** cross-encoder with RRF gave the best result on every top-of-list metric: MRR 0.936, nDCG 0.919, lenient R@1 0.90, in 0.5–1.9 s on CPU. Re-runs gave the same numbers. This is now the recommended default: `RERANK_STRATEGY=cross_fused`, `RERANK_MODEL=cross-encoder/ms-marco-MiniLM-L-6-v2`.

4.5 LLM listwise re-ranking and the silent-failure incident

I added an LLM re-ranker behind an OpenAI-compatible client so the provider is just config (Groq, Ollama, Cerebras). Getting it working was the messy part. Cerebras returned 402 (payment required). Ollama locally was too slow on 16 GB RAM. On Groq, `llama-3.3-70b-versatile` had been retired (404), and `gpt-oss-120b` returned **empty replies** because its reasoning tokens used up the small `max_tokens` budget.

The dangerous part: the re-ranker fell back to vector order on failure, so the eval still produced plausible numbers while about 70 s per question went to silent retries. The fix was engineering rather than modelling. I made 400/401/403/404 errors non-retryable (fail fast), gave `gpt-oss` `max_tokens ≥ 2000` with `reasoning_effort='low'`, treated empty text as a failure, added a STATS counter printed in every report, and added a warning when more than 20% of calls fail. The final run had 37/37 calls succeed: MRR 0.894 and the best sR@5 (0.967), but at about 20 s per query it is not competitive with MiniLM.

Lesson: a fallback path can quietly turn an experiment into a copy of the baseline. Count failures and surface them next to the metrics.

4.6 Cleaning footers and the noise floor

The handbook repeats a page footer 41 times, and these lines pollute chunks. My first cleaning rules also deleted real content (the degree-classification table), so I tightened them. A line is now removed only if it has a leading or trailing page number *and* repeats 5+ times. `rechunk.py` re-chunked from stored raw text in one transaction (288 chunks).

Surprisingly, clean chunks scored slightly *lower* (baseline MRR 0.861 vs 0.876). The cleaning was correct. The drop came from shifted chunk boundaries splitting some evidence quotes differently. That shift moved MRR by about 0.05, the same size as the re-ranking gains. So the honest conclusion is that **with 30 questions, differences under ~0.05 are noise**. MiniLM+RRF still beat the baseline on clean chunks (0.886 vs 0.861), so the direction holds.

4.7 Abstention is not a threshold

Mean top similarity was 0.73 for answerable questions vs 0.66 for unanswerable ones. The distributions overlap too much for a score threshold alone to decide 'I don't know'. Abstention belongs in the answer phase (an LLM checking whether the evidence supports an answer), not in retrieval.

5. Lessons

- **Measure on your own data.** The bigger re-ranker lost to the baseline; the smaller one won.
- **Fuse, don't replace.** RRF over ranks makes a re-ranker safe: it can only promote chunks that retrieval also found plausible.
- **Debug by elimination.** Batch vs single, sanity pair, then per-question inspection. Each script ruled out one hypothesis.
- **Instrument the failure path.** Silent fallbacks produce believable wrong numbers. Count and report every failed call.
- **Know your noise floor.** A chunk-boundary change moved metrics as much as the model did, so the eval set has to grow before finer claims are made.
- **Latency is a metric.** 0.936 MRR in 1 s beats 0.894 in 20 s for a user-facing product.

6. How I tell it in an interview

"I built a RAG retrieval stack on pgvector and wrote an eval harness with strict and lenient evidence matching, MRR and nDCG. The obvious move, adding a large cross-encoder, dropped MRR from 0.88 to 0.66. I proved it wasn't a bug, found it was over-weighting lexical overlap on handbook boilerplate, and fixed it with Reciprocal Rank Fusion. A small MiniLM re-ranker with RRF then gave MRR 0.94 in about a second. Along the way I caught an LLM re-ranker silently failing and falling back to baseline, and made the pipeline fail fast and report its failure rate. I also found the eval's noise floor was about 0.05, so my next step is a bigger eval set with bootstrap confidence intervals."

7. What's next

Next step	Why
Bootstrap confidence intervals in run.py	Tell real gains from ± 0.05 noise
Grow eval set (synthetic + 2nd document)	More questions shrink the noise floor; tests generalisation
Hybrid search: BM25 + vector with RRF	Exact-term queries (codes, names) where embeddings are weak
Set MiniLM-fused as default in .env	Best measured quality/latency trade-off
Answer phase (/ask) with grounded citations + abstention	Retrieval quality only matters once answers use it